

Dynamic Economic-Denial-of-Sustainability (EDoS) Detection in SDN-based Cloud

Phuc Trinh Dinh¹, and Minho Park^{*1,2}

¹Department of Information Communication, Materials, and Chemistry Convergence Technology,
Soongsil University, Seoul 156-743, South Korea

²School of Electronic Engineering, Soongsil University, Seoul 156-743, South Korea
phuctrinh@soongsil.ac.kr, and mhp@ssu.ac.kr

Abstract—In Cloud Computing, a new type of attack, called Economic Denial of Sustainability (EDoS) attack, exploits the pay-per-use model to scale up the resource usage over time to the extent that the cloud user has to pay for the unexpected usage charge. To prevent EDoS attacks, we propose an efficient solution in the SDN-based cloud computing environment. In this paper, we first apply an unsupervised learning approach called Long Short-Term Memory (LSTM), which is a multivariate time series anomaly detection, to detect EDoS attacks. Its key idea is to try to predict values of the resource usage of a cloud consumer (CPU load, memory usage and etc). Furthermore, unlike other existing proposals using a predefined threshold to classify the anomalies which generate high rate errors, in this work, we utilize a dynamic error threshold which delivers much better performance. Through practical experiments, the proposed EDoS attack defender is proven to outperform existing mechanisms for EDoS attack detection. Furthermore, it also outperforms some of the machine-learning-based methods, which we conducted the experiment ourselves. The comprehensive experiments conducted with various EDoS attack levels prove that the proposed mechanism is an effective, innovative approach to defense EDoS attacks in the SDN-based cloud.

Index Terms—Cloud computing, economic denial of sustainability (EDoS), software-defined networking, machine learning, network function virtualization, cloud computing security.

I. INTRODUCTION

Cloud Computing offers some of the exclusive features such as payment according to usage, elasticity, auto scalability, which are considered an opportune demand over traditional computing services. However, Cloud Computing is prone to a number of attacks [1]. Especially, due to a pay-per-use pricing model, Economic Denial of Sustainability (EDoS) [6], a new variant of Distributed Denial of Service (DDoS) attacks, is emerging as one of the biggest challenges.

A cloud service provider generally auto-scales up the resources for a customer to satisfy the service level agreements (SLA) for the availability of the service for the customer and charges the customer for the use. EDoS attacks exploit this feature to make the cloud unsustainable by fading the cloud billing mechanism to charge the cloud user's bill for the attack's activities. Since EDoS is relatively new and stealthy, it is more challenging to detect EDoS compared to DDoS. Therefore, only a few existing proposals have an effective solution to tackle the problem [6].

Some of the existing mitigation techniques [7], [8], [10] focused exclusively on graphical Turing test or crypto-puzzle based verifier to distinguish human from machine. However, these kinds of techniques often have time complexity which is unfeasible to deploy in real life. Notwithstanding these techniques, many researchers used a predefined threshold value setting that might lead to a higher false-negative rate [6]. Recently, a few new approaches such as the queuing model, statistical have been applied to tackle EDoS attack [12], [13]. However, according to the authors, their approaches primarily still have their own limitations such as lacking evaluation metrics and generating high rate errors. Understanding these difficulties, in our paper, we propose a framework to effectively detect EDoS attacks which is a multivariate time-series anomaly detection in which its main mechanism is LSTM. Furthermore, unlike most of other existing mitigation techniques [7]–[11] focused solely on setting a predefined statics threshold value which increases both false positive and negative rate [12], here we introduce a dynamic error threshold technique to automatically set the optimal threshold. In summary, our major contributions are as follows:

- We propose a scheme to effectively detect EDoS attacks by applying a multivariate time series anomaly EDoS detection through the LSTM-based technique, and then using the Dynamic Error Threshold to automatically set the optimal threshold to distinguish between normal and abnormal traffic.
- We investigate common types of EDoS attacks in conventional clouds and present a new perspective of EDoS attacks in the SDN environment.
- Our experiments are conducted in a real cloud computing environment using OpenStack [19] integrated with SDN environment using ONOS controller [25], but it could be effortlessly applied to other controllers.

The rest of the piece of the paper is organized as follows: In section II, we present some related works. The background knowledge is described in section III. In section IV, we explain in detail our proposed scheme. The experimental setup and performance evaluation will be delivered in sections V and VI, respectively. Finally, conclusion and future researches are mentioned in Section VII.

* Corresponding author

II. RELATED WORK

A number of studies on EDoS mitigation have been conducted since the protection of EDoS cloud customers has become vital. EDoS-Shield [7] is mitigation first proposed by Squalli et al.. Two main components, virtual firewall (VFs) and authentication nodes (V-nodes), perform a major role to detect EDoS attacks. Although EDoS-Shield is reasonably practical but, nonetheless, there are major drawbacks of the method proposed. Particularly, it is easily susceptible to spoofing attacks and possesses a higher false-positive rate by blocking many legal user IP addresses since the two lists are not updated in an efficient manner. sPoW [8] is another EDoS mitigation technique which its main function is to filter illegitimate traffic. There are few shortcomings of the scheme. Firstly, an attacker can easily start a puzzle accumulation attack without attempting to solve crypto-puzzle based requests. Secondly, the scheme may be prone to false-positive rates as some of the benign users might be prevented from the service request due to rising puzzle complexity. EDoS Armor [9] uses two-phase defense system, consists of admission control and congestion control. The aim of admission control is to restrict the number of end-users. Congestion control sets priorities to the client based on their past browsing behavior which is categorized as good and bad users utilizing decision tree algorithm (J48). In general, EDoS Armor is relatively effectual, however, the adaptability of the model needs to take into account because potential new clients might lose interest using the site if it is too complicated.

Recently, a few new approaches such as the queuing model, statistical analysis have been applied to tackle EDoS attacks. Chowdhury et al. proposed a new solution known as EDoS Eye employing game theory to mitigate the invalid traffic [12]. The authors obtained optimal threshold value through the Nash equilibrium and incorporated honeypot to minimize false rates. Another EDoS detection scheme proposed by Karami & Chen [13]. This scheme is an anomaly-based detection system in which Markov models were obtained. According to the authors, the technique primarily aimed at identifying EDoS traffic instead of mitigation.

To the best of our knowledge, our work is the first proposal applied machine-learning based approach which achieves state-of-the-art results in SDN-based cloud aimed to mitigate EDoS attacks.

III. BACKGROUND KNOWLEDGE

A. SDN-BASED CLOUD

The integration of Network Function Virtualization and Software Defined Networking technologies is indicated as the SDN/NFV architecture [2], as illustrated in Figure 1. It consists of NFV Orchestration, Controller Platform, forwarding devices, and servers. SDN controller is responsible for controlling the traffic path using primarily OpenFlow protocol to communicate with forwarding devices (OpenFlow switch) to impose policies from the control plane to the data plane. Meanwhile, the NFV uses standard computing virtualization technology to consolidate in commodity hardware

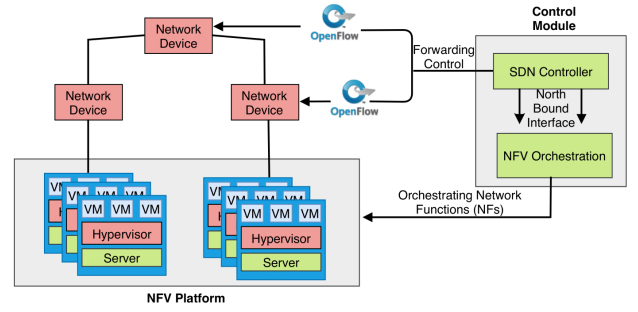


Figure 1: SDN-based cloud design

(i.e., servers or cloud platform (e.g OpenStack)) to deliver Network functions of high bandwidth and high performance with low cost. Hypervisors, which run on servers, primarily focus on supporting VMs that enable them to operate Network Functions such as Firewall, Proxies, IDS.

The logical control module is composed of the SDN controller [3] and the NFV orchestration system. The NFV orchestration system is responsible for provisioning virtualized network functions and is controlled through standard interfaces or APIs by the SDN controller. After determining the policy requirements and creating network topology, the Controller generates optimal function assignments and assigns the functions to certain VMs in the optimized path by translating the logic policy specifications which can be known as a service chain. The NFV orchestration system conducts a service function chain and the controller instructs the traffic through the required and appropriate sequence of VMs by installing network flow rules into forwarding devices.

B. EDOS ATTACKS IN SDN-BASED CLOUD

To diminish EDoS effectively, we understand that knowing the characteristics of EDoS is important. Because of that we briefly compare DDoS and EDoS attacks in the cloud. In DDoS attack [4], [5], the attacker's goal is to disrupt the services offered by a cloud service provider. Therefore, in most of the cases, DDoS attackers irrationally launch an

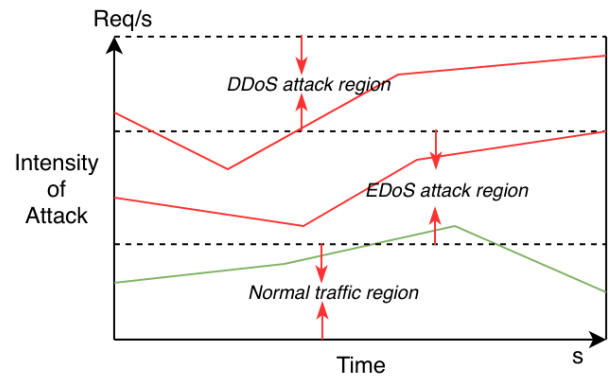


Figure 2: EDoS attack region with attack strength

TABLE I: EDoS attacks in SDN-Based Cloud

Attack	EDoS Simulator
TCP SYN flooding attack	Bonesi [26]
TCP (HTTP) flooding attack	Bonesi
UDP flooding attack	Bonesi
Slowloris	slowloris.py
Low and Slow rate attack	R.U.D.Y.py

attack over a short amount of time with their maximum resources. The purpose of such an attack is to shut down the service completely or significantly degrade the service quality. Conversely, in EDoS attacks, attackers are typically more clever and highly rational. They particularly target an individual or specific group of cloud users. They gradually push illegitimate traffic over a longer period of time. Their motivation is to increase the financial cost incurred by the targeted cloud user through the over-provisioning of cloud resources. Figure 2 illustrates the difference in traffic patterns between EDoS and DDoS. EDoS is very stealthy in nature and remains below the threshold value of a typical intrusion detection system so that it can pass the filter like legitimate traffic. In conclusion, it is more difficult to detect EDoS than DDoS, and it requires the right technique to detect EDoS attacks effectively.

For the viewpoint of SDN, which is a flow-based rule network system, types of EDoS attack are listed in Table I. These volumetric-based attacks not only make cloud user but also SDN controller or Virtual Network Function consuming a lot of resources (e.g. CPU, RAM and so on).

C. BASICS OF LONG SHORT-TERM MEMORY

Due to the complex temporal dependence and stochasticity of multivariate time-series data such as the cloud consumers' data. Even a well-known neural network in deep learning called Recurrent Neural Network (RNN) [15] where the output from the previous step is fed as input to the current step is able to represent the time dependence, but it fails to learn the long-term dependence in a sequence. Fortunately, a variant form of RNN which is Long Short-Term Memory (LSTM), was proposed to address the problem [14] owing to the introduction of a weighted self-loop conditioned on the context that enables them to forget past knowledge in addition to accumulating it. In addition, LSTM approaches have been shown to model complex nonlinear feature interaction [16] which are often present in multivariate time-series data streams, and obviate the need to specify a time-window in which to consider data values in an anomaly detection task due to the use of shared parameters cross-time [15].

IV. A PROPOSED EDOS ATTACK DEFENSE

In this section, we first introduce the objective of the proposed scheme. Next, the workflow system will be presented. Finally, the internal modules will be explained in detail.

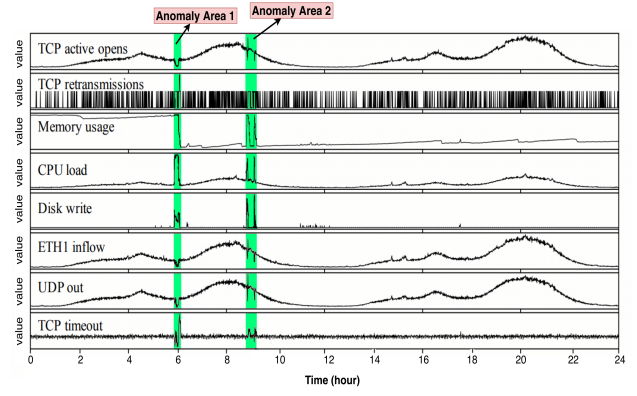


Figure 3: Multivariate time series snippet visualization, with two anomalous regions highlighted in green

A. AN OBJECTIVE AND SYSTEM DESIGN ANALYSIS

In the SDN-based cloud, cloud consumers are typically monitored with multivariate time series, whose anomaly detection is critical for service quality management. For example, as shown in Figure 3, networking metrics such as Memory usage, CPU load, TCP connections and etc are supervised by a network administrator, and when an EDoS attack is launched, one of the selected features will suddenly change their values. Thus, we need an intelligent model to discover any changes in values of multiple time series simultaneously to detect EDoS. Therefore, in this paper, an algorithm called LSTM is adopted for multivariate time series anomaly detection. By learning from historical data and then using live data to predict future values, we compare the values with a dynamic error threshold to determine anomalies. Therefore, we claim that our proposed scheme is very a practical and effective method to detect EDoS attacks.

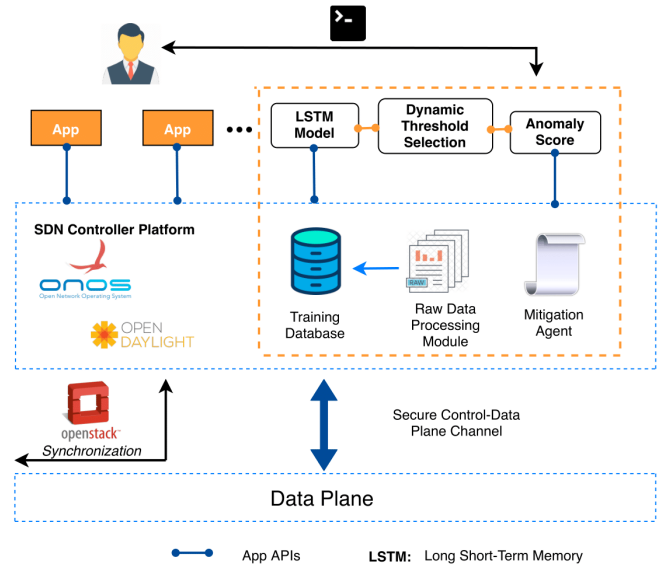


Figure 4: Conceptual Architecture

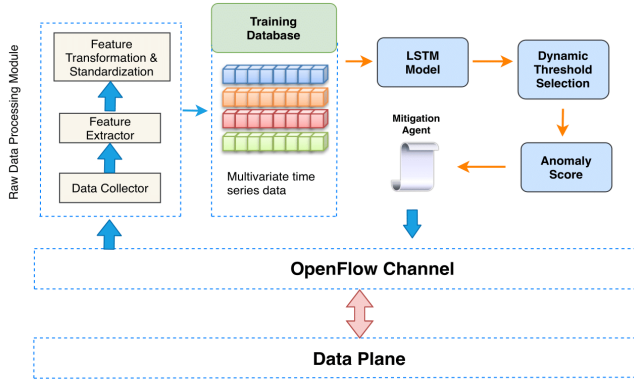


Figure 5: Detailed architecture modules synchronized OpenStack controller located in the SDN application layer

Figure 4. depicts an overview of our proposed framework aimed to detect EDoS attack in Software-Defined Networking architecture synchronized with OpenStack [19] controller and it is an extension in the Control Plane. This framework includes extension modules that can be implemented and distributed both in the cloud controller platform (OpenStack) and SDN application plane. We recommend that these modules should be placed at a dedicated security server in real deployment in order to reduce the SDN controller processing load. For convenience, however, in this work we design modules including LSTM Model, Dynamic Error Threshold and Anomaly Score Classifier and locate these modules in the SDN application layer, just some modules are placed in the OpenStack controller for synchronization and further actions. Note that, our proposed scheme can be deployed in a distributed manner of nowadays cloud computing, in which databases can be shared among distributed agents to enhance the efficiency of anomaly detection.

B. A WORKFLOW SYSTEM

In this work, our mechanism is situated in the SDN controller and consists of six separate modules: Raw Data Processing (Data Collector, Feature Extractor and Feature Transformation and Normalization), LSTM Model, Dynamic Threshold Selection, Anomaly Result, and Mitigation Agent as shown in Figure 5. During data preprocessing, the Data Collector reliably collects data on the utilization of the physical and virtual resources comprising deployed clouds and accumulates flow information from OpenFlow switches in a predetermined interval time. Next, this data will be then sent to the Feature Extractor to extract data's attributes. Subsequently, the data is transformed by data standardization, and then it is stored in the Training Database module. After the preprocessing, the stored data which represents a multivariate time-series representation in the Training Database is sent to an unsupervised LSTM training model module (more details in section IV.C) to predict a single scalar prediction $\hat{y}^{(t)}$ for the actual value at each step t . Once a predicted value $\hat{y}^{(t)}$ is generated, the prediction error is calculated for each step t . Next, the Dynamic Threshold Selection module sets an anomaly threshold dynamically and

the Anomaly Score module outputs an anomaly score. By comparing the anomaly score with the threshold, the network traffic will be classified as anomalous or normal. If the traffic is declared as normal traffic, then it will be normally forwarded to the destination; if not, the traffic will be transferred to the Mitigation Agent which performs a dropping or deleting action if it detects abnormal traffic.

Note that, in the proposed scheme, the Training Database is continually updated by data's attributes collected from the above loop. In a preset time which is being defined by a network administrator, the LSTM Model will be trained by using the updated database. By doing so, the proposed mechanism can well adapt to the various network systems.

C. INTERNAL MODULES

Let us have a closer look into the internal modules of our proposed mechanism.

- **DATA COLLECTOR:** this module simply runs in the SDN controller. It collects data from both OpenFlow statistics and the utilization rate of the physical and virtual resources comprising deployed clouds.
- **FEATURE EXTRACTOR:** it extracts data information from Data Collector to take out several attributes, as shown in Table II. These features are key features which are selected by selecting k highest scores based on our empirical research and are verified by using chi-squared statistic [20].
- **FEATURE TRANSFORMATION & STANDARDIZATION:** at this stage, all the categorical features, which are extracted in the previous stage, are transformed from categorical data into numeric by using One-Hot encoding technique. After the transformation, the data needs to be normalized, and its formula can be expressed as:

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

where x is the value that is standardized, μ is the mean of the distribution and σ is the standard deviation of the distribution. This module not only significantly improves the LSTM Model's predictive power, but also offers them for faster to run and more easily understood.

- **LSTM MODEL:** consider a time series $X = x^{(1)}, x^{(2)}, \dots, x^{(n)}$ where each step $x^{(t)} \in R^m$ in the time series is an m -dimensional vector $x_1^{(t)}, x_2^{(t)}, \dots, x_m^{(t)}$, whose elements correspond to input variable [15]. For

TABLE II: Key Features

Feature	Description
protocol_type	network protocol
no. of bytes	number of bytes
no. of packets	number of packets
flow_duration	flow duration
network_bandwidth	network bandwidth
duration	length (number of seconds) of the connection
cpu_load	CPU usage
memory_usage	memory usage

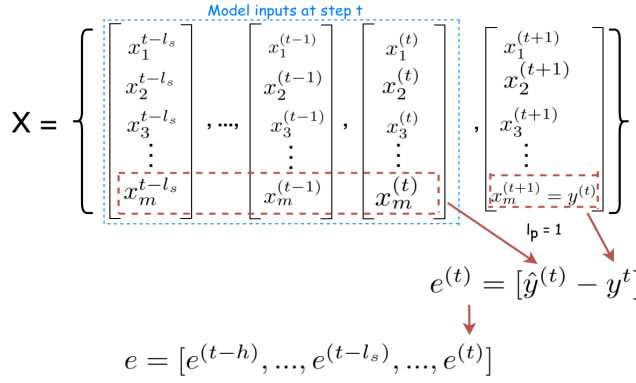


Figure 6: A visual representation of the input matrices used for prediction at each time step t . Current prediction errors are compared to past errors to determine if they are anomalous

each point $x^{(t)}$, a sequence length l_s decides the number of points to input into the model for prediction. A prediction length l_p then decides the number of steps ahead to predict, where the number of dimensions d being predicted is $1 \leq d \leq m$. Since our goal is to predict where $d = 1$. The $l_p = 1$ is utilized to restrict the number of predictions for each step t and reduce processing time. As a result, a single scalar prediction $\hat{y}^{(t)}$ is generated for the actual value at each step t (as showed in Figure 6). In situations where either $l_p > 1$ or $d > 1$ or both, Gaussian parameters can be used to represent matrices of predicted values at a single step t [15].

Once a predicted values $\hat{y}^t$ is generated for each step t , the prediction error is calculated as $e^{(t)} = |y^{(t)} - \hat{y}^{(t)}|$, where $y^{(t)} = x_i^{(t+1)}$ with i corresponding to the dimension of the true value (see Figure 6). Each $e^{(t)}$ is appended to a one-dimensional vector of errors:

$$\mathbf{e} = [e^{(t-h)}, \dots, e^{(t-l_s)}, \dots, e^{(t-1)}, e^{(t)}]$$

where h is the number of historical error values used to evaluate current errors.

- **DYNAMIC THRESHOLD SELECTION:** we adopt an unsupervised method that achieves high performance with low overhead and without the use of labeled data or statistical assumptions about errors that were proposed in [17]. With a threshold ϵ selected from the set:

$$\epsilon = \mu(\mathbf{e}_s) + \mathbf{z}\sigma(\mathbf{e}_s)$$

where ϵ is determined by:

$$\epsilon = \argmax(\epsilon) = \frac{\Delta\mu(\mathbf{e}_s)/\mu(\mathbf{e}_s) + (\Delta\sigma(\mathbf{e}_s)/\sigma(\mathbf{e}_s))}{|\mathbf{e}_a| + |\mathbf{E}_{seq}|^2}$$

such that:

$$\Delta\mu(\mathbf{e}_s) = \mu(\mathbf{e}_s) - \mu(\{e_s \in \mathbf{e}_s | e_s < \epsilon\})$$

$$\Delta\sigma(\mathbf{e}_s) = \sigma(\mathbf{e}_s) - \sigma(\{e_s \in \mathbf{e}_s | e_s < \epsilon\})$$

$$\mathbf{e}_a = \{e_s \in \mathbf{e}_s | e_s > \epsilon\}$$

$$\mathbf{E}_{seq} = \text{continuous sequences of } e_a \in \mathbf{e}_a$$

- **ANOMALY SCORE:** an exponentially-weighted average (EWMA) [18] is adopted in this paper to generate the smoothed errors $\mathbf{e}_s = [e_s^{(t-h)}, \dots, e_s^{(t-l_s)}, \dots, e_s^{(t-1)}, e_s^{(t)}]$ [17]. To evaluate whether values are nominal, we set a threshold for their smoothed prediction errors - values corresponding to smoothed errors above the threshold are classified as anomalies.

Values evaluated for ϵ are decided using $z \in \mathbf{z}$ where $\mathbf{z}$ is an ordered set of positive values representing the number of standard deviations above $\mu(\mathbf{e}_s)$. Values for $\mathbf{z}$ rely on context, however, empirically, values work well in a range of between two and ten. Values for $\mathbf{z}$ less than two generally resulted in too many false positives. Once $\argmax(\epsilon)$ is decided, each resulting anomalous sequence of smoothed errors $\mathbf{e}_{seq} \in \mathbf{E}_{seq}$ outputs an anomaly score, s , indicating the severity of the anomaly:

$$s^{(i)} = \frac{\max(\mathbf{e}_{seq}^{(i)}) - \argmax(\epsilon)}{\mu(\mathbf{e}_s) + \sigma(\mathbf{e}_s)}$$

In simple terms, a threshold is found that, if all values above are omitted, would cause the greatest percent decrease in the mean and standard deviation of the smoothed error $\mathbf{e}_s$ [17]. The function also penalizes for having larger number of anomalous values ($|\mathbf{e}_a|$) and sequences ($|\mathbf{e}_{seq}|$) in order to avoid overly greedy behavior. Then the highest smoothed error in each sequence of anomalous errors is given a normalized score based on its distance from the chosen threshold.

- **MITIGATION AGENT:** if the traffic is declared as abnormal traffic, then, in order to mitigate the EDoS attack, the mitigation Agent sends a *flow_mod* message with a *delete* action to the edge OpenFlow switch and requests the forwarding engine of the SDN controller to drop *packet_in* messages of attacking sources (as illustrated in Figure 5).

V. EXPERIMENT SETUP

First, this section gives readers a description to train LSTM. Second, we present an elaborate implementation of the proposed scheme.

A. MODEL SETUP

Server Machine Dataset (SMD) is used to train the LSTM. This new dataset was collected from a large Internet company, and it was publicly published in [21]. The SMD includes an anomaly traffic with the ratio of 10.72% and the training & testing set size is 58317 and 73729, respectively.

To train the LSTM, we strictly follow the instructions in [17]. In addition, the model parameters are described in Table III.

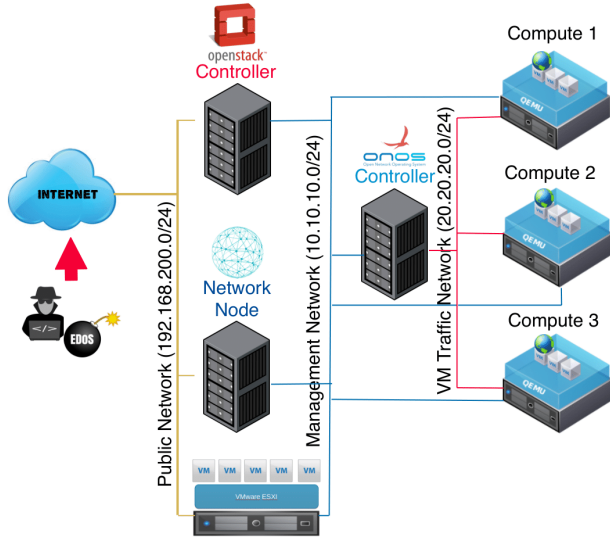


Figure 7: Experimental topology

B. SDN-BASE CLOUD IMPLEMENTATION AND TEST PREPARATION

In this paper, we followed and deployed OpenStack-SDN based test-bed for EDoS as proposed in [22]. Figure 7 shows the testbed for virtual networks to simulate EDoS attack in production environment which comprises of a OpenStack platform (one Controller node, one Network node, one VMware ESXI server to create multiple VMs, and three Compute nodes running OvS drivers for cloud networking), a SDN controller (ONOS), the Internet connection and botnets. The victim machine was flooded with different kinds of EDoS attacks using different simulators (as shown in Table I in which *slowloris.py* and *R.U.D.Y.py* are coded by us), besides sFlow-RT [27] was utilized to collect data. As we stated in section IV.a, our modules locate in the SDN application layer of the ONOS server (Figure 7). The ONOS server system configurations are dodeca-core CPU (Intel(R) Core(TM) i7-8700 CPU @ 3.20GHz) and a total 48GB memory, running 64 bit Ubuntu Linux v18.04. Note that, we will evaluate our proposed scheme on this server.

VI. PERFORMANCE EVALUATION

First, we will compare our EDoS attack defender with another anomalous machine-learning-based detection algorithm, which is One-Class SVM [24]. This algorithm is unsupervised

TABLE III: Model Parameters

hidden layers	2
units in hidden layers	80
sequence length (l_s)	250
training iterations	30
dropout	0.2
batch size	64
optimizer	Adam

learning which is similar to our algorithm, however, it is a simpler technique. Second, to the best of our knowledge, the work proposed in [11] is the only existing work that evaluates their mechanism using two evaluation metrics: Response Time and CPU Usage which are essential criteria to evaluate the EDoS defense mechanism [6]. Hence in this paper, our proposed scheme will be compared with Zubair A.Baig et al. 's proposal.

To properly evaluate the performance of our proposed scheme, we closely follow and adopt some particular criteria such as Detection Rate, Accuracy and False Alarm Rate which are proposed in [23]. Besides, Response Time is also an important aspect to consider a good EDoS attack detection mechanism [6]. The criteria are calculated by using 4 distinct measurements, True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN). Specifically, True Positive is the probability of abnormal traffic which is classified as illegitimate traffic, True Negative shows the probability of legit traffic that is trusted as legitimate traffic, False Positive reflects the probability of normal traffic that is accepted as

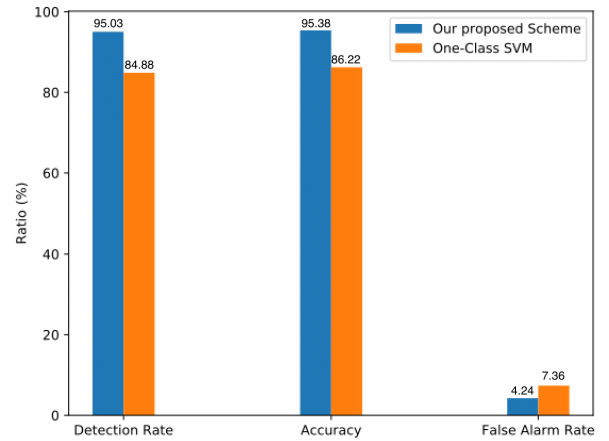


Figure 8: The Detection Rate, Accuracy, FAR comparison

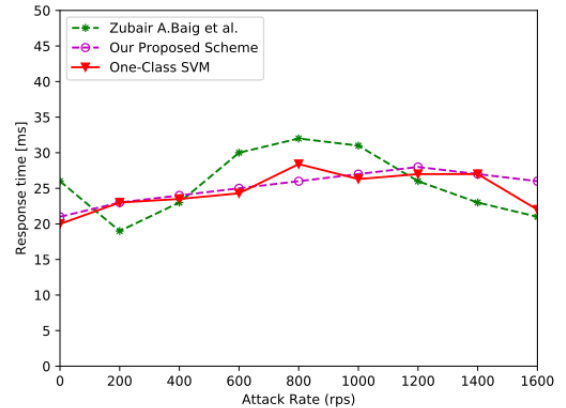


Figure 9: The experimental result of the response time

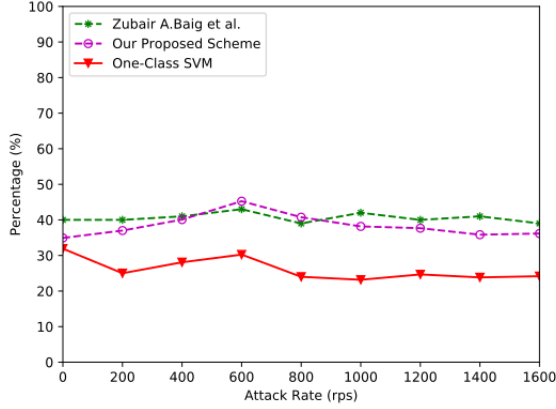


Figure 10: The percentage of the CPU utilization

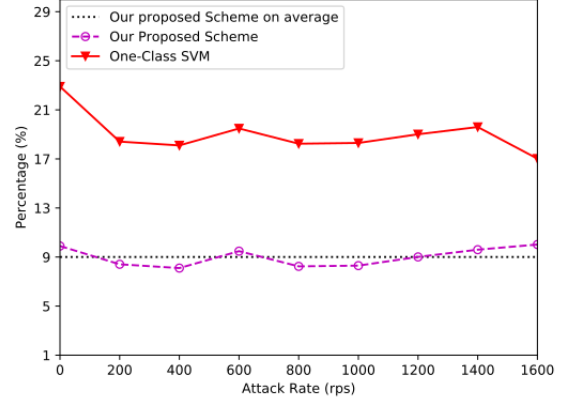


Figure 11: The percentage of the memory utilization

abnormal traffic and False Negative reveals the probability of illegal traffic that are recognized as legit traffic.

- Detection Rate (Sensitivity/TPR): is the ratio of the number of detected malicious flows to the number of all malicious flows:

$$TPR = \frac{TP}{TP + FN} \quad (2)$$

- Accuracy (AC): shows the percentage of true detection over total the number of flows:

$$AC = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

- False Positive Rate (FPR): is the proportion of abnormal flows incorrectly identified as normal flows:

$$FPR = \frac{FP}{FP + TN} \quad (4)$$

Based on the above formulas and Table IV, the Detection, Accuracy and False Alarm Rate are calculated respectively as illustrated in Figure 8. Here, regarding the Detection Rate and Accuracy, our novel approach outperforms the One-Class SVM mechanism when they always account for the highest rate, 95.03%, and 95.38%, respectively. Whereas, the One-Class SVM respectively occupies 84.88% and 86.22% in term of both the ratio of detection and the accuracy. In respect of the False Alarm Rate, our scheme also presents domination in producing the wrong warning when only occupies 4.24%, while the rate of incorrect alarm generated by the One-Class SVM is around 7.36%. Overall, the One-Class SVM algorithm not only produces a lower detection rate and accuracy but also causes a higher false alarm rate.

Next, Figure 9 reveals the Response Time comparison between our proposed scheme, One-Class SVM and [11].

TABLE IV: Confusion Matrix

TP (%)	TN (%)	FP (%)	FN (%)
95.76	94.99	4.24	5.01

Owing to the lightweight of the Dynamic Error Threshold module of our mechanism, the end to end response time of our proposed scheme is less, which only slightly fluctuates between 21ms and 28ms. However, due to a simpler design, One-Class SVM performs better than our proposed scheme a bit. Whereas, the Zubair A.Baig et al' work creates much higher response time.

In Figure 10, the resource consumption evaluation is also conducted in which the percentage of CPU usage of our proposed scheme and Zubair A.Baig et al' work are around 38.43%, 41%, respectively. With respect to memory utilization, our proposed scheme only consumes around 9.06% of the memory on average, meanwhile, the One-Class method occupies much higher memory on average compared to our mechanism, as shown in Figure 11.

Overall, our proposed scheme outperforms both the Zubair A.Baig et al' proposal and the One-Class SVM.

VII. CONCLUSION AND FUTURE WORK

In this paper, we propose an unsupervised learning approach called Long Short-Term Memory (LSTM), which is a multivariate time series anomaly detection integrated with a dynamic error threshold, to mitigate EDoS attacks in SDN-Based cloud environment. The evaluation in section VI shows that our proposed scheme is very efficient in both terms of accuracy and resource consumption. As our future work, we expect to improve the mechanism and compare our proposal with other existing works using more evaluation criteria.

ACKNOWLEDGEMENT

This work was supported by Institute for Information & communications Technology Promotion(IITP) grant funded by the Korea government(MSIT) (No.2018-0-00254, SDN security technology development).

REFERENCES

- [1] Q. Yan, F. R. Yu, Q. Gong and J. Li, "Software-Defined Networking (SDN) and Distributed Denial of Service (DDoS) Attacks in Cloud Computing Environments: A Survey, Some Research Issues, and Challenges," in *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 602-622, Firstquarter 2016.
- [2] Y. Li and M. Chen, "Software-Defined Network Function Virtualization: A Survey," in *IEEE Access*, vol. 3, pp. 2542-2553, 2015.
- [3] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," in *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14-76, Jan. 2015.
- [4] T. V. Phan and M. Park, "Efficient Distributed Denial-of-Service Attack Defense in SDN-Based Cloud," in *IEEE Access*, vol. 7, pp. 18701-18714, 2019.
- [5] S. Dong, K. Abbas and R. Jain, "A Survey on Distributed Denial of Service (DDoS) Attacks in SDN and Cloud Computing Environments," in *IEEE Access*, vol. 7, pp. 80813-80828, 2019.
- [6] F. Z. Chowdhury, L. B. M. Kiah and M. A. M. Ahsan, "Economic denial of sustainability (EDoS) mitigation approaches in cloud: Analysis and open challenges," 2017 ICECOS, Palembang, 2017, pp. 206-211.
- [7] M. H. Sqalli, F. Al-Haidari and K. Salah, "EDoS-Shield - A Two-Steps Mitigation Technique against EDoS Attacks in Cloud Computing," 2011 Fourth IEEE International Conference on Utility and Cloud Computing, Victoria, NSW, 2011, pp. 49-56.
- [8] Khor, S.H. and A. Nakao, "spow: On-demand cloud-based eddos mitigation mechanism," in *HotDep (Fifth Workshop on Hot Topics in System Dependability)*. 2009.
- [9] M. Masood, Z. Anwar, S. A. Raza and M. A. Hur, "EDoS Armor: A cost effective economic denial of sustainability attack mitigation framework for e-commerce applications in cloud environments," *INMIC*, Lahore, 2013, pp. 37-42.
- [10] M. Naresh Kumar, P. Sujatha, V. Kalva, R. Nagori, A. K. Katukojwala and M. Kumar, "Mitigating Economic Denial of Sustainability (EDoS) in Cloud Computing Using In-cloud Scrubber Service," 2012 Fourth International Conference on Computational Intelligence and Communication Networks, Mathura, 2012, pp. 535-539.
- [11] Zubair A.Baig, Sadiq M.Sait, Farid Binbeshr, "Controlled access to cloud resources for mitigating Economic Denial of Sustainability (EDoS) attacks,".
- [12] F. Z. Chowdhury, M. Y. I. Idris, M. L. M. Kiah and M. A. M. Ahsan, "EDoS eye: A game theoretic approach to mitigate economic denial of sustainability attack in cloud computing," 2017 ICSGRC, Shah Alam, 2017, pp. 164-169.
- [13] Karami, M. and S. Chen. "Attribution of Economic Denial of Sustainability Attacks in Public Clouds," in *International Conference on Security and Privacy in Communication Systems*. 2016. Springer.
- [14] Junyoung Chung, C. Gulcehre, K. Cho, Y. Bengio, "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," *arXiv:1412.3555 [cs.NE]*.
- [15] Pankaj Malhotra, L. Vig, G. Shroff, P. Agarwal, "Long short term memory networks for anomaly detection in time series", *ESANN 2015 proceedings, Computational Intelligence and Machine Learning*.
- [16] O. Ogunmolu, X. Gu, S. Jiang, N. Gans, "Nonlinear Systems Identification Using Deep Dynamic Neural Networks", *arXiv:arXiv:1610.01439*.
- [17] K. Hundman, V. Constantinou, C. Laporte, Ian Colwell, Tom Soderstrom, "Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding," *arXiv:1802.04431 [cs.LG]*.
- [18] J. Stuart Hunter, "The exponentially weighted moving average," *Journal of quality technology*, 1986 - Taylor & Francis.
- [19] Welcome to OpenStack Documentation. [Online]. Available: <http://docs.openstack.org/>
- [20] A. Tamilarasan, S. Mukkamala, A. H. Sung and K. Yendrapalli, "Feature Ranking and Selection for Intrusion Detection Using Artificial Neural Networks and Statistical Methods," *The 2006 IEEE International Joint Conference on Neural Network Proceedings*, Vancouver, BC, 2006, pp. 4754-4761.
- [21] Ya Su, Y. Zhao, C. Niu, R. Liu, W. Sun and Dan Pei, "Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network," in *25th ACM SIGKDD conference on knowledge discovery and data mining*.
- [22] P. Singh and S. Manickam, "Design and deployment of OpenStack-SDN based test-bed for EDoS," 2015 4th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO) (Trends and Future Directions), Noida, 2015, pp. 1-5.
- [23] Nicolas Goix, "How to Evaluate the Quality of Unsupervised Anomaly Detection Algorithms?," <https://arxiv.org/abs/1607.01152>
- [24] Kun-Lun Li, Hou-Kuan Huang, Sheng-Feng Tian and Wei Xu, "Improving one-class SVM for anomaly detection," *Proceedings of the 2003 International Conference on Machine Learning and Cybernetics (IEEE Cat. No.03EX693)*, Xi'an, 2003, pp. 3077-3081 Vol.5.
- [25] ONOS, "ONOS: towards an open, distributed SDN OS," *HotSDN '14 Proceedings of the third workshop on Hot topics in software defined networking*, <https://onosproject.org/>.
- [26] BoNeSi, "The DDoS Botnet Simulator," Accessed 2015 [Online], Available: <https://github.com/markus-go/bonesi>.
- [27] sFlow-RT, <http://www.inmon.com>, May 20, 2014.